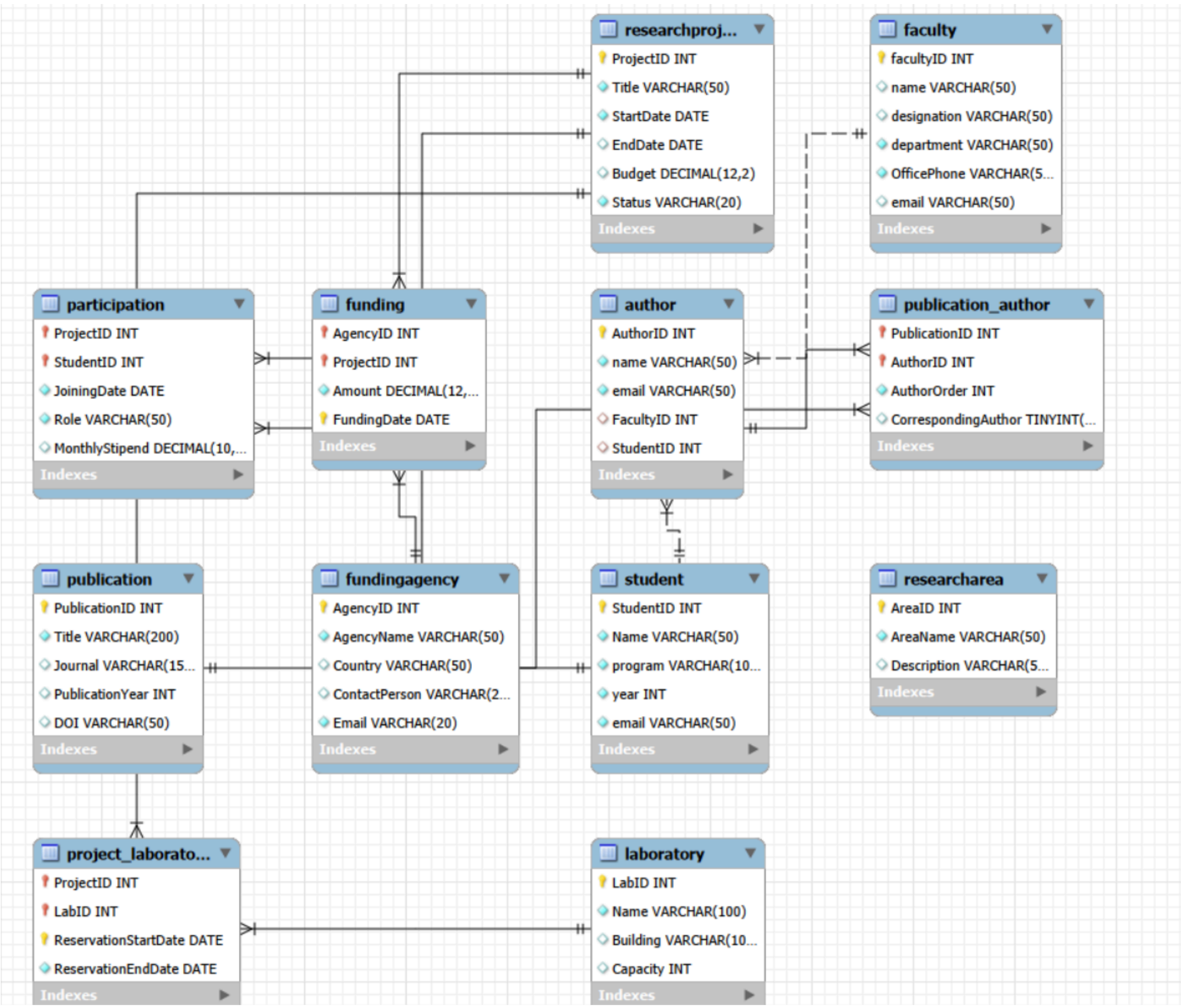
1.(b). Convert the ER model into relational database schema
For each relation specify Primary Key, Foreign Keys, Composite Keys (if any):



Q. 1(c) Write SQL CREATE TABLE statements for any six important tables including Primary Keys, Foreign Keys, Appropriate data types, NOT NULL constraints.

1. **ResearchArea := (base parent for others)**

```
create table ResearchArea(
  AreaID int primary key unique,
  AreaName varchar(50) not null,
  Description varchar(50)
);
```

Result Grid | Filter Rows: | Export: | Wr

	Field	Type	Null	Key	Default	Extra
▶	AreaID	int	NO	PRI	NULL	
	AreaName	varchar(50)	NO		NULL	
	Description	varchar(50)	YES		NULL	

2. **Research_Project := (Table with a 1:N Foreign Key referencing ResearchArea)**

```
create table ResearchProject(
  ProjectID int primary key,
  Title varchar(50) not null,
  StartDate DATE not null,
  EndDate DATE,
  Budget Decimal(12,2),
  Status Varchar(20) not null,
  FacultyID int not null, -- foreignK from the Faculty --
  AreaID int not null , -- foreignK from the ReasearchArea--

  Foreign Key (FacultyID) REFERENCES Faculty(FacultyID),
  Foreign Key (AreaID) REFERENCES ResearchArea(AreaID)
);
```

	Field	Type	Null	Key	Default	Extra
	Title	varchar(50)	NO		NULL	
	StartDate	date	NO		NULL	
	EndDate	date	YES		NULL	
	Budget	decimal(12,2)	YES		NULL	
	Status	varchar(20)	NO		NULL	

3. **Participation := (Associative table with multiple PK and FK i.e junction table)**

```
create table Participation(
  ProjectID int not null,
  StudentID int not null,
  JoiningDate DATE not null,
  Role varchar(50) not null,
  MonthlyStipend decimal(10,2),
  primary key(ProjectID,StudentID),
  Foreign Key (ProjectID) REFERENCES ResearchProject(ProjectID),
  Foreign Key (StudentID) REFERENCES Student(StudentID)
);
```

	Field	Type	Null	Key	Default	Extra
▶	ProjectID	int	NO	PRI	NULL	
	StudentID	int	NO	PRI	NULL	
	JoiningDate	date	NO		NULL	
	Role	varchar(50)	NO		NULL	
	MonthlyStipend	decimal(10,2)	YES		NULL	

4. **Publication_Author := (associative table with additional constraint)**

```
create table Publication_Author(
  PublicationID int not null,
  AuthorID int not null,
  AuthorOrder int not null,
  CorrespondingAuthor Boolean Default false,
  PRIMARY KEY (PublicationID,AuthorID),
  Foreign Key (PublicationID) REFERENCES Publication(PublicationID),
  Foreign Key (AuthorID) REFERENCES Author(AuthorID),
  unique(PublicationID,AuthorOrder),
  check(AuthorOrder > 0)
);
```

Field	Type	Null	Key	Default	Extra
PublicationID	int	NO	PRI	NULL	
AuthorID	int	NO	PRI	NULL	
AuthorOrder	int	NO		NULL	
CorrespondingAuthor	tinyint(1)	YES		0	

5. **Faculty := (Base parent Table that serves has PK serving FK for multiple tables)**

```
create table Faculty(
  FacultyID int primary key,
  Name varchar(50) not null,
  Designation varchar(50),
  Department varchar(50) not null,
  OfficePhone varchar(50) not null,
  Email varchar(50) not null unique
);
```

Field	Type	Null	Key	Default	Extra
facultyID	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
designation	varchar(50)	YES		NULL	
department	varchar(50)	NO		NULL	
OfficePhone	varchar(50)	NO		NULL	
email	varchar(50)	YES		NULL	

6. **Project_Laboratory (associative table with additional constraints)**

```
CREATE TABLE Project_Laboratory (
   ProjectID INT NOT NULL,
   LabID INT NOT NULL,
   ReservationStartDate DATE NOT NULL,
   ReservationEndDate DATE NOT NULL,
   PRIMARY KEY (ProjectID, LabID, ReservationStartDate),
   FOREIGN KEY (ProjectID)
      REFERENCES ResearchProject(ProjectID),
   FOREIGN KEY (LabID)
      REFERENCES Laboratory(LabID),
   CHECK (ReservationEndDate >= ReservationStartDate)
);
```

Field	Type	Null	Key	Default	Extra
ProjectID	int	NO	PRI	NULL	
LabID	int	NO	PRI	NULL	
ReservationStartDate	date	NO	PRI	NULL	
ReservationEndDate	date	NO		NULL	

2. A training institute stores student enrollment information in a single spreadsheet. The current table is given where:
• One student enrolls in many courses
• One course has many students.
• One course is taught by one instructor in a semester.
• One instructor teaches many courses.
• One textbook is prescribed for one course.
• One textbook may be used by multiple courses.

Q. 2(a) Normalize the database into Third Normal Form (3NF). And design a relational database. Create a relational schema which should include the following information:

In order to reach the 3NF form for this database schema , the following steps are needed to be performed:

Step 1: Identify Functional Dependencies (FDs)

Based on the rules and sample data in the image:

1. studentID-> studentName,program
2. InstructorID -> InstructorName, InstructorOffice
3. TextbookISBN -> TextbookTitle, Publisher
4. CourseID-> CourseName,TextBookISBN (since 1 textbook is given per course)
5. studentID,CourseID,Semester -> InstructorID ,grade (determines who teaches that std section and their grade as well)

Step 2: Remove Partial & Transitive Dependencies

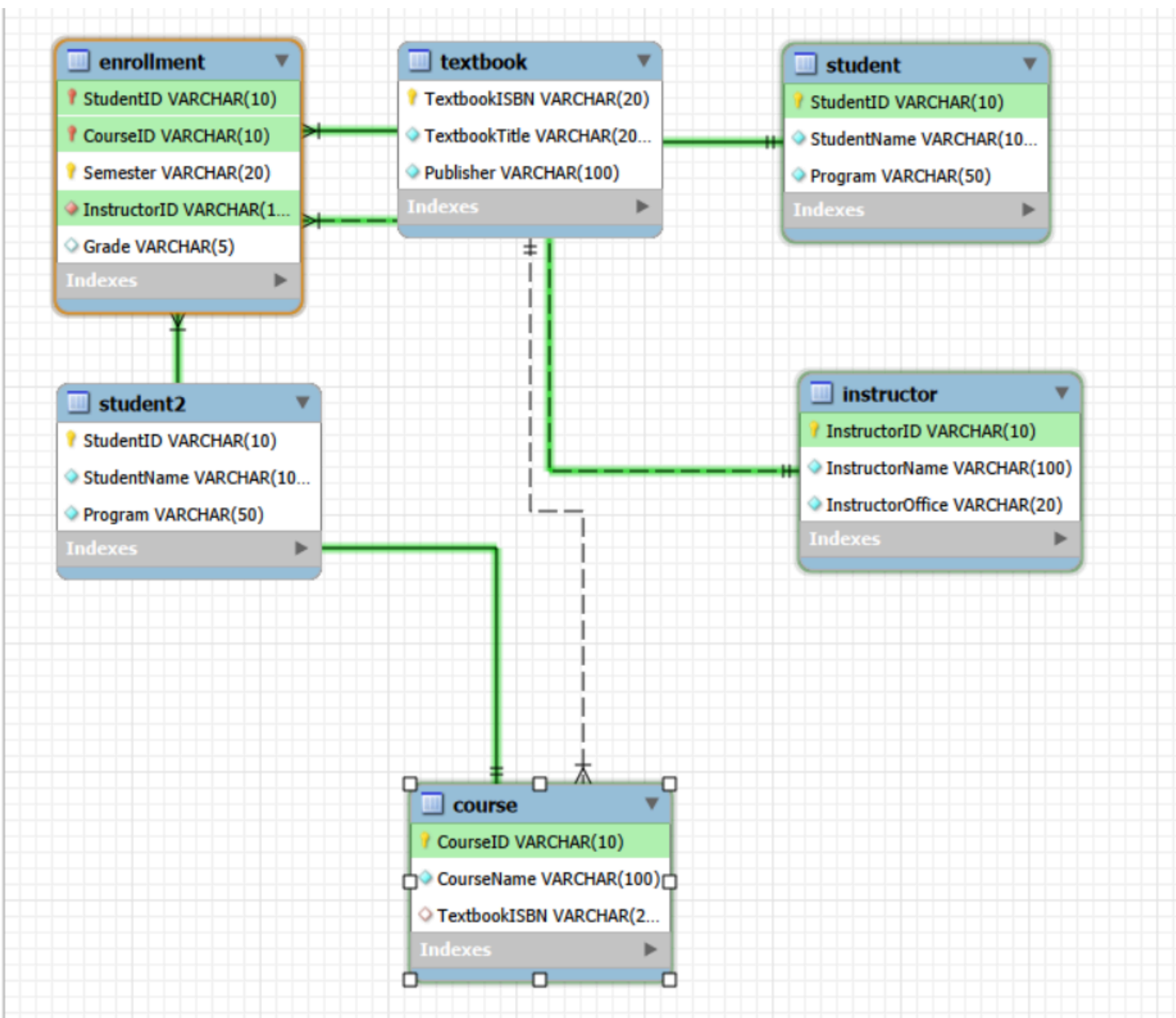
To reach 3NF, we must ensure:

- 1NF: All values are atomic (already met).
- 2NF: Every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).
- 3NF: No non-key attribute is transitively dependent on a primary key (no non-key attribute determines another non-key attribute).

We split the original unnormalized relation into 5 distinct 3NF relations:

Table Name	Attributes (underlined one are primary keys and last one is composite)
Student	<u>studentID</u>,StudentName,Program
Instructor	<u>InstructorID</u>,InstructorName,InstructorOffice
TextBook	<u>TextBookISBN</u>,TextBookTitle,Publisher
Course	<u>CourseID</u>,CourseName,TextBookISBN
Enrollment	<u>StudentID,CourseID,semester</u>, InstructorID,Grade

The given is the EER diagram for these schema representation



Q. 2(b) Write SQL CREATE TABLE statements for creating tables including Primary Keys, Foreign Keys, Appropriate data types, NOT NULL constraints.

```
create database Lab7second;
use  lab7second;
```

```
CREATE TABLE STUDENT2 (
   StudentID VARCHAR(10) PRIMARY KEY,
   StudentName VARCHAR(100) NOT NULL
   Program VARCHAR(50) NOT NULL
);
describe student;
```

Field	Type	Null	Key	Default	Extra
StudentID	varchar(10)	NO	PRI	NULL	
StudentName	varchar(100)	NO		NULL	
Program	varchar(50)	NO		NULL	

```
CREATE TABLE INSTRUCTOR (
   InstructorID VARCHAR(10) PRIMARY KEY,
   InstructorName VARCHAR(100) NOT NULL,
   InstructorOffice VARCHAR(20) NOT NULL
);
describe instructor;
```

Field	Type	Null	Key	Default	Extra
InstructorID	varchar(10)	NO	PRI	NULL	
InstructorName	varchar(100)	NO		NULL	
InstructorOffice	varchar(20)	NO		NULL	

```
CREATE TABLE TEXTBOOK (
   TextbookISBN VARCHAR(20) PRIMARY KEY,
   TextbookTitle VARCHAR(200) NOT NULL,
   Publisher VARCHAR(100) NOT NULL
);
describe textbook;
```

Field	Type	Null	Key	Default	Extra
TextbookISBN	varchar(20)	NO	PRI	NULL	
TextbookTitle	varchar(200)	NO		NULL	
Publisher	varchar(100)	NO		NULL	

```
CREATE TABLE COURSE (
   CourseID VARCHAR(10) PRIMARY KEY,
   CourseName VARCHAR(100) NOT NULL,
   TextbookISBN VARCHAR(20),
   FOREIGN KEY (TextbookISBN)
 REFERENCES TEXTBOOK(TextbookISBN)
);
describe course;
```

Field	Type	Null	Key	Default	Extra
CourseID	varchar(10)	NO	PRI	NULL	
CourseName	varchar(100)	NO		NULL	
TextbookISBN	varchar(20)	YES	MUL	NULL	

```
CREATE TABLE ENROLLMENT (
   StudentID VARCHAR(10),
   CourseID VARCHAR(10),
   Semester VARCHAR(20),
   InstructorID VARCHAR(10) NOT NULL,
   Grade VARCHAR(5),
   PRIMARY KEY (StudentID, CourseID, Semester),
   FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
   FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID),
   FOREIGN KEY (InstructorID) REFERENCES INSTRUCTOR(InstructorID)
);
describe enrollment;
```

Field	Type	Null	Key	Default	Extra
StudentID	varchar(10)	NO	PRI	NULL	
CourseID	varchar(10)	NO	PRI	NULL	
Semester	varchar(20)	NO	PRI	NULL	
InstructorID	varchar(10)	NO	MUL	NULL	
Grade	varchar(5)	YES		NULL	